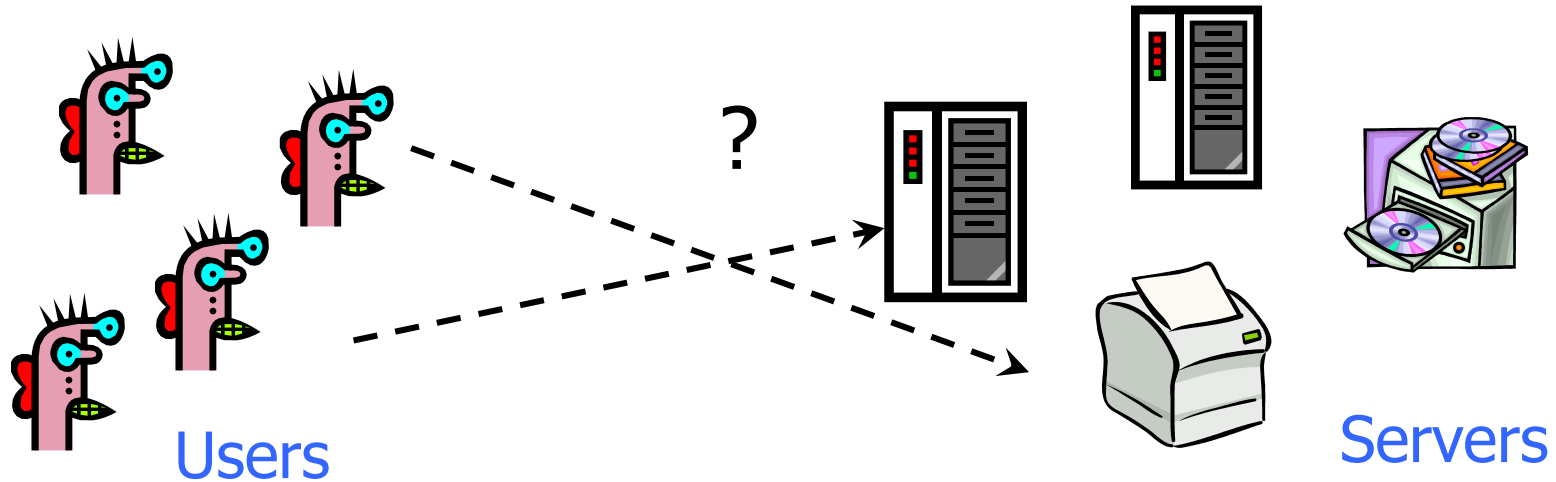


Many-to-Many Authentication



How do users prove their identities when requesting services from machines on the network?

Naïve solution: every server knows every user's password

- **Insecure:** break into one server $\Rightarrow$ compromise all users
- **Inefficient:** to change password, user must contact every server

Requirements

u Security

- ... against attacks by passive eavesdroppers and actively malicious users

u Transparency

- Users shouldn't notice authentication taking place
- Entering password is Ok, if done rarely

u Scalability

- Large number of users and servers

Threats

u User impersonation

- Malicious user with access to a workstation pretends to be another user from the same workstation

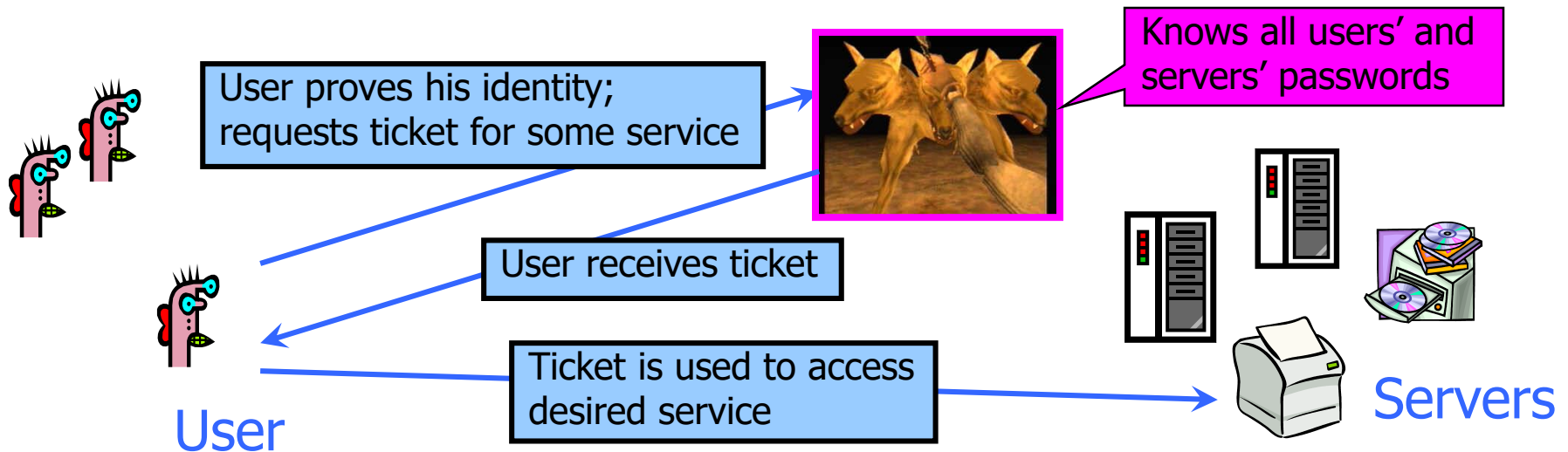
u Network address impersonation

- Malicious user changes network address of his workstation to impersonate another workstation

u Eavesdropping, tampering, replay

- Malicious user eavesdrops, tampers, or replays other users' conversations to gain unauthorized access

Solution: Trusted Third Party



- u Trusted **authentication service** on the network
 - Knows all passwords, can grant access to any server
 - Convenient (but also the single point of failure!)
 - Requires high level of physical security

Needham-Schroeder Shared-Key Protocol

- Parties: A, B, and trusted server T
- Setup: A and T share K_{AT} , B and T share K_{BT}
- Goal: Mutual entity authentication between A and B; key establishment
- Messages:

$A \rightarrow T: A, B, N_A$ (1)

$A \leftarrow T: E[K_{AT}] (N_A, B, k, E[K_{BT}](k, A))$ (2)

$A \rightarrow B: E[K_{BT}] (k, A)$ (3)

$A \leftarrow B: E[k] (N_B)$ (4)

$A \rightarrow B: E[k] (N_B - 1)$ (5)

What bad things can happen if there is no N_A ?

Another subtle flaw in Step 3.

Kerberos Overview

- One issue of Needham-Schroeder – Needs $[K_{AT}]$ for every communication.
- Kerberos solution:
 - Separates TTP into an AS and a TGS.
- The client authenticates to AS using a long-term *shared secret* and receives a TGT [SSO].
- Use this TGT to get additional tickets from TGS without resorting to using the shared secret.

AS = Authentication Server
SS = Service Server

TGS = Ticket Granting Server
TGT = Ticket Granting Ticket

Kerberos Protocol – 2 (Simplified)

1. $C \rightarrow AS: TGS \parallel N_C$

2. $AS \rightarrow C: \{K_{C,TGS} \parallel C\}_{K_{AS,TGS}} \parallel \{K_{C,TGS} \parallel N_C \parallel TGS\}_{K_{AS,C}}$

(Note that the **first** part of message 2 is the **ticket granting ticket (TGT)** for the TGS)

3. $C \rightarrow TGS: SS \parallel N'_C \parallel \{K_{C,TGS} \parallel C\}_{K_{AS,TGS}} \parallel \{C \parallel T_1\}_{K_{C,TGS}}$

4. $TGS \rightarrow C: \{K_{C,SS} \parallel C\}_{K_{TGS,SS}} \parallel \{K_{C,SS} \parallel N'_C \parallel SS\}_{K_{C,TGS}}$

(Note that the **first** part in message 4 is the **ticket** for the server S).

5. $C \rightarrow SS: \{K_{C,SS} \parallel C\}_{K_{TGS,SS}} \parallel \{C \parallel T_2\}_{K_{C,SS}}$

6. $SS \rightarrow C: \{T_3\}_{K_{C,SS}}$

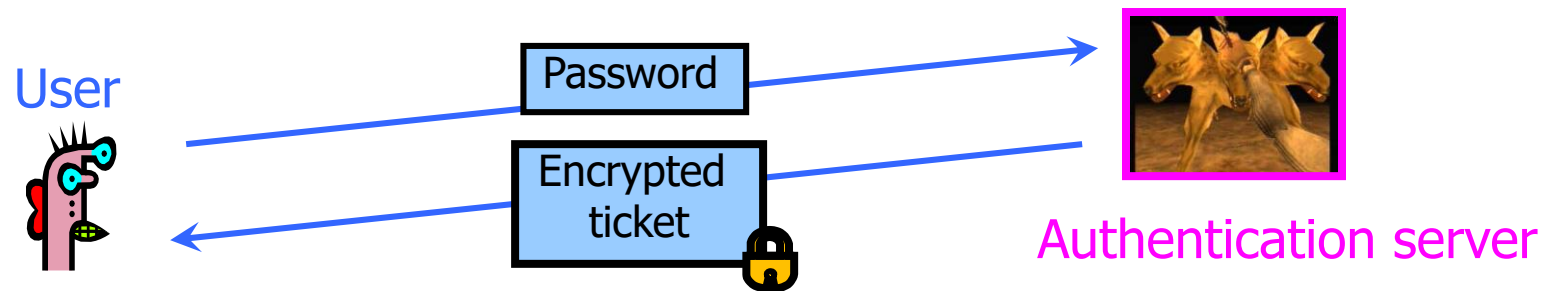
Kerberos Overview

- One issue of Needham-Schroeder – Needs $[K_{AT}]$ for every communication.
- Kerberos solution:
 - Separates TTP into an AS and a TGS.
- The client authenticates to AS using a long-term *shared secret* and receives a TGT [SSO].
- Use this TGT to get additional tickets from TGS without resorting to using the shared secret.

AS = Authentication Server
SS = Service Server

TGS = Ticket Granting Server
TGT = Ticket Granting Ticket

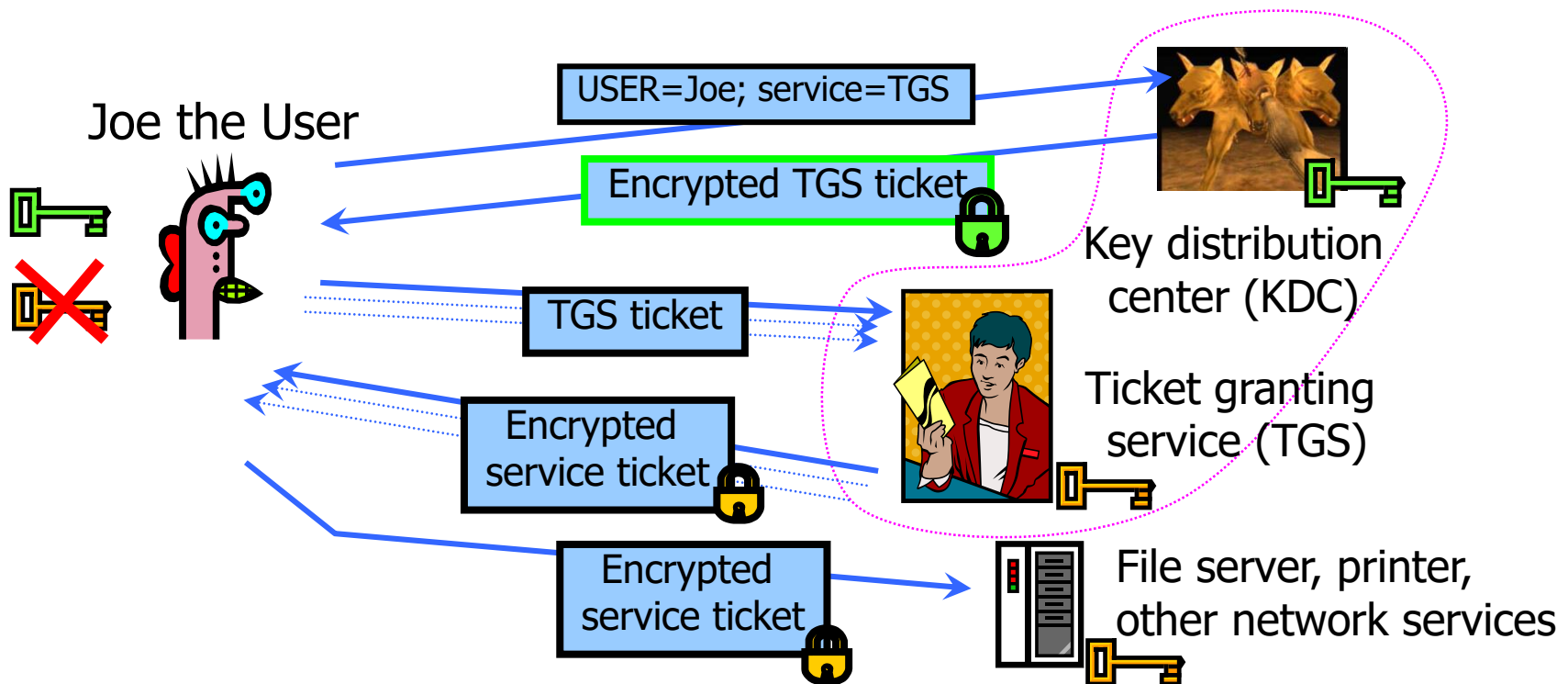
Naïve Authentication



- u **Insecure:** passwords are sent in plaintext
 - Eavesdropper can steal the password and later impersonate the user to the authentication server
- u **Inconvenient:** need to send the password each time to obtain the ticket for any network service
 - Separate authentication for email, printing, etc.

Two-Step Authentication

- u Prove identity once to obtain a special TGS ticket
- u Use TGS to get tickets for any network service



Threats

u Ticket hijacking

- Malicious user may steal the service ticket of another user on the same workstation and try to use it
 - Network address verification does not help
- Servers must verify that the user who is presenting the ticket is the same user to whom the ticket was issued

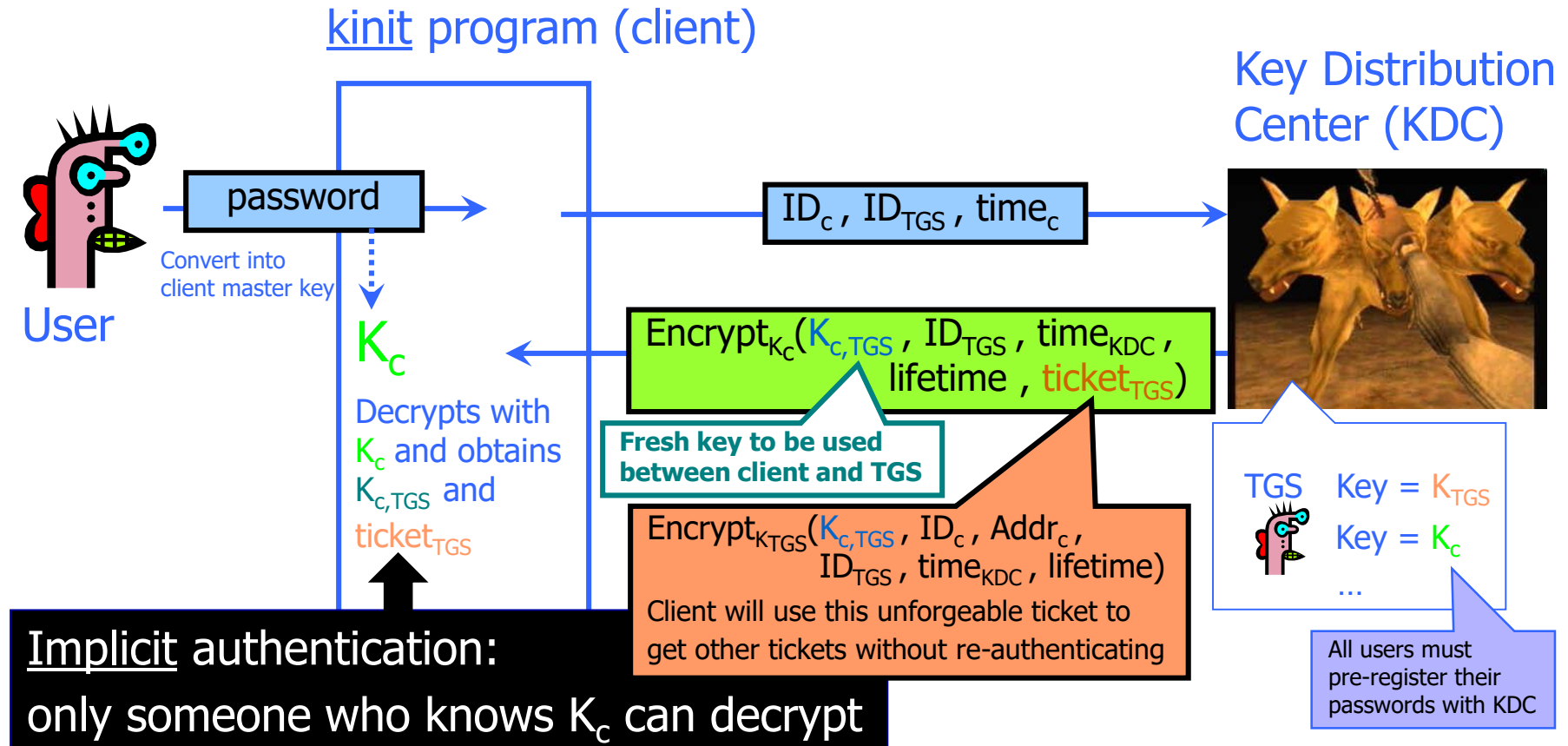
u No server authentication

- Attacker may misconfigure the network so that he receives messages addressed to a legitimate server
 - Capture private information from users and/or deny service
- Servers must prove their identity to users

Symmetric Keys in Kerberos

- u K_C is long-term key of client C
 - Derived from the user's password
 - Known to the client and the key distribution center (KDC)
- u K_{TGS} is long-term key of TGS
 - Known to KDC and the ticket granting service (TGS)
- u K_V is long-term key of network service V
 - Known to V and TGS; each service V has its own long-term key
- u $K_{C,TGS}$ is short-term session key betw. C and TGS
 - Created by KDC, known to C and TGS
- u $K_{C,V}$ is short-term session key between C and V
 - Created by TGS, known to C and V

"Single Logon" Authentication



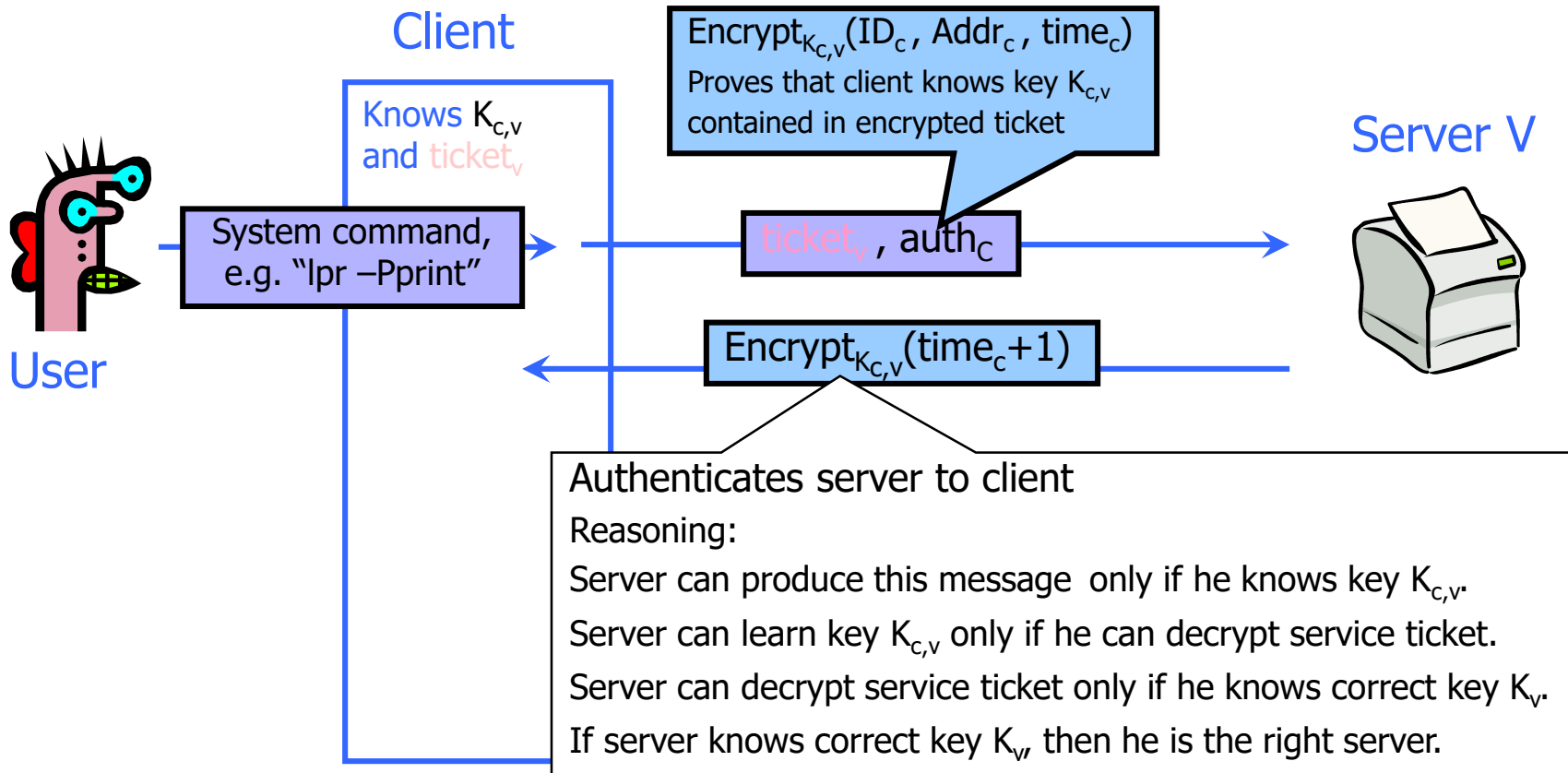
- u Client only needs to obtain TGS ticket **once** (say, every morning)
- u Ticket is encrypted; client cannot forge it or tamper with it

© 2006 The Authors
Journal compilation © 2006 Blackwell Publishing Ltd



- slide 13

Obtaining Service

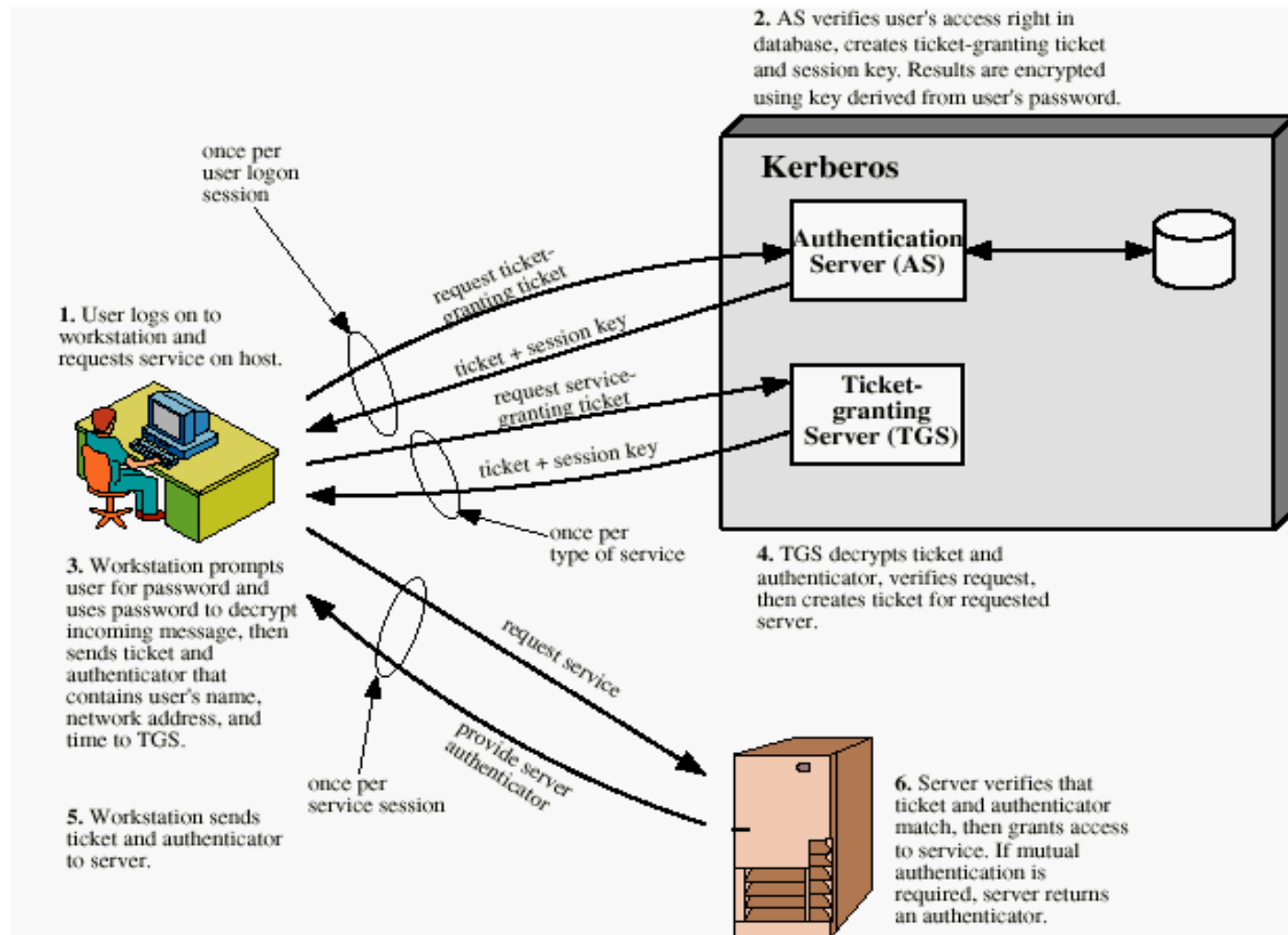


- u For each service request, client uses the short-term key for that service and the ticket he received from TGS

Kerberos in Large Networks

- u One KDC isn't enough for large networks
- u Network is divided into **realms**
 - KDCs in different realms have different key databases
- u To access a service in another realm, users must...
 - Get ticket for home-realm TGS from home-realm KDC
 - Get ticket for remote-realm TGS from home-realm TGS
 - As if remote-realm TGS were just another network service
 - Get ticket for remote service from that realm's TGS
 - Use remote-realm ticket to access service
 - $N(N-1)/2$ key exchanges for full N-realm interoperoperation

Summary of Kerberos



Important Ideas in Kerberos

u Short-term session keys

- Long-term secrets used only to derive short-term keys
- Separate session key for each user-server pair
 - Re-used by multiple sessions between same user and server

u Proofs of identity based on authenticators

- Client encrypts his identity, addr, time with session key; knowledge of key proves client has authenticated to KDC
 - Also prevents replays (if clocks are globally synchronized)
- Server learns this key separately (via encrypted ticket that client can't decrypt), verifies client's authenticator

u Symmetric cryptography only

Kerberos Version 5

- u Better user-server authentication
 - Separate subkey for each user-server session instead of re-using the session key contained in the ticket
 - Authentication via subkeys, not timestamp increments
- u Authentication forwarding (delegation)
 - Servers can access other servers on user's behalf, eg, can tell printer to fetch email
- u Realm hierarchies for inter-realm authentication
- u Explicit integrity checking + standard CBC mode
- u Multiple encryption schemes, not just DES

Practical Uses of Kerberos

- u Microsoft Windows
- u Email, FTP, network file systems, many other applications have been **kerberized**
 - Use of Kerberos is transparent for the end user
 - Transparency is important for usability!
- u Local authentication
 - login and su in OpenBSD
- u Authentication for network protocols
 - rlogin, rsh
- u Secure windowing systems

Reading

- u Kaufman Chapters 13 and 14
- u “Designing an Authentication System: A Dialogue in Four Scenes”
 - A high-level survey of network threats and design principles behind Kerberos

Kerberos Drawback

- Single point of failure:
- Security partially depends on tight clock synchronization.
- Useful primarily inside an organization
 - Does it scale to Internet? What is the main difficulty?

Acknowledgment

u Slides from Vitaly Shmatikov